



Module 13-1

Using Functions and Tasks

cādence®

This module describes Verilog subroutines, known as functions and tasks. It explains where and how to define them, how to invoke them, and some issues involved with using them.

Module Objective

In this module, you:

- Write Verilog subroutines to encapsulate functionality making your code more readable and reusable

Topics

- Verilog Subroutine Introduction
- Functions – Declaration and Calling
- Tasks – Declaration and Calling
- Issues with Functions and Tasks



Your objective is to effectively use subroutines to encapsulate functionality and thus to make your code more readable and reusable. To do that, you need to know what subroutines are available and how to use them.

Verilog Subroutines Introduction

Subroutines:

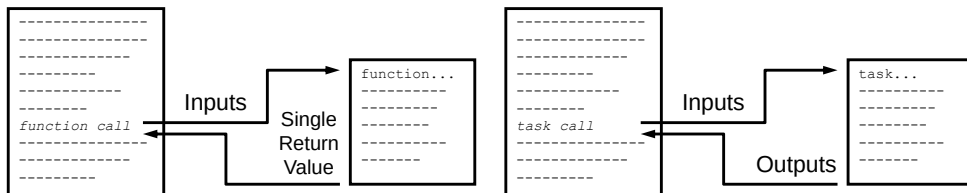
- Encapsulate code that might otherwise be duplicated.
- Contain statements that execute in sequence.

Function subroutines:

- Have one or more inputs and return a single value.
- Are invoked as an expression term.

Task subroutines:

- Have zero or more input/outputs.
- Are invoked as a procedural statement.



Let's first examine the purpose of subroutines:

- You can think of a subroutine as a level of hierarchy, or structure, in sequential code.
- You use subroutines to encapsulate some frequently used code in one place where you can more easily manage it. You can provide this code with inputs, to which you assign values as you invoke the subroutine. These input values can flexibly direct the subroutine to perform slightly differently for each invocation.
- Encapsulating sequential code in subroutines is similar to encapsulating design units in modules.

Declaring Functions

- Function declared only within a module
- Starts with **function** keyword
- Then the logic description of what the function does
- Ends with **endfunction** keyword
- Alternate syntax with the function port list

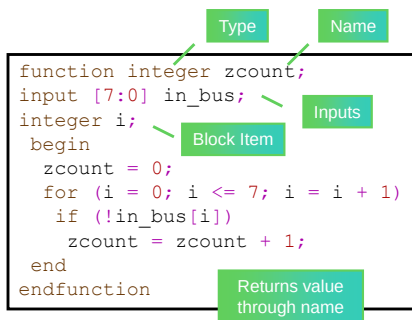


Diagram illustrating the standard Verilog function syntax with annotations:

```
function integer zcount;
input [7:0] in_bus;
integer i;
begin
  zcount = 0;
  for (i = 0; i <= 7; i = i + 1)
    if (!in_bus[i])
      zcount = zcount + 1;
end
endfunction
```

Annotations:

- Type: integer
- Name: zcount
- Inputs: in_bus
- Block Item: zcount = 0;
- Returns value through name: zcount

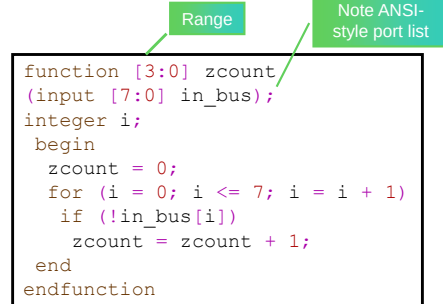


Diagram illustrating the alternate Verilog function syntax (ANSI-style port list) with annotations:

```
function [3:0] zcount
(input [7:0] in_bus);
integer i;
begin
  zcount = 0;
  for (i = 0; i <= 7; i = i + 1)
    if (!in_bus[i])
      zcount = zcount + 1;
end
endfunction
```

Annotations:

- Range: [3:0]
- Note ANSI-style port list: (input [7:0] in_bus);

Syntax

```
function [automatic] [signed] [range_or_type] function_identifier;
  tf_input_declaration;
  {tf_input_declaration;}
  {block_item_declaration}
  function_statement
endfunction
```

Syntax Alternative

```
function [automatic] [signed] [range_or_type] function_identifier
(function_port_list);
  {block_item_declaration}
  function_statement
endfunction
```

You declare a function only within a module. The declaration starts with the function keyword.

Functions are by default static. That means that exactly one copy of the function variables exists. All calls to the function utilize that one set of function variables, thus calls to static functions are generally not recursive. External code can access static function variables using hierarchical references. Verilog 2001 added the automatic declaration. Automatic functions create a new temporary set of their variables for each call. Automatic functions can be recursive. External code cannot access automatic function variables.

The return type defaults to a single bit. You can alternatively specify integer, real, realtime, or time, or a vector range. A vector is by default not signed. You can declare the vector signed. The signed declaration is a Verilog 2001 feature.

A function must have at least one input port and must have no output ports and no inout ports. You can declare the input ports using either syntax. With the function port list syntax, you can prefix port attributes. The input port type defaults to a single bit. You can alternatively specify integer, real, realtime, or time, or a vector range. A vector is by default not signed. You can declare the vector signed. The signed declaration is a Verilog 2001 feature.

You can declare additional block items. You cannot declare module items. For example, you can declare variables but not nets.

A function declaration contains a statement. The statement may be a statement block, for example grouped between the begin and end keywords. Functions cannot invoke the scheduler. That means function assignments are always blocking and functions always execute completely and instantly.

Functions can have side-effects, for example, a function can assign to a module variable but not to a module net. For a function to have side-effects is undesirable programming practice. Instead, declare the function return vector sufficiently wide to hold all the data you need to return, then deconstruct the vector value as needed upon return.

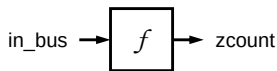
A function declaration ends with the endfunction keyword.

Calling Functions

You call a function as an expression term.

```
function_identifier (expression {,expression})
```

```
function integer zcount (  
    input [7:0] in_bus  
);  
    integer i;  
    begin  
        zcount = 0;  
        for (i=0; i<=7; i=i+1)  
            if (in_bus[i]==0)  
                zcount = zcount + 1;  
        end  
    endfunction
```



```
module zfunc (  
    input [7:0] a_bus, b_bus,  
    output [31:0] account,  
    output reg bzero  
);  
  
    // function declaration here  
  
    assign account = zcount(a_bus);  
  
    always @(b_bus)  
        bzero = zcount(b_bus)==0;  
  
endmodule
```

You call a function as an expression term. The simulator assigns the values of the argument expressions to the input ports in the order in which they appear in the call.

When the function completes, the simulator uses the value of the function name variable where the function call appears in the calling expression. Functions return a single value that you use as an operand in an rvalue expression. An rvalue expression is one that may appear only on the right side of an assignment – you cannot assign a value to an rvalue expression. If the function never completes, for example, it executes a loop that loops forever, it never returns and the simulation “hangs”.

In this example, the zcount function returns an integer value representing the number of bits of the in_bus port that are zero. It returns a value between 0 and 8. The continuous assignment call assigns the returned value to the “account” module output port. The procedural assignment call uses the function return value as an operand in a comparison expression.

Constant Functions

Verilog-1995 allows simple constant expressions for defining limits for vectors, replicates, etc.

- Verilog-2001 allows limits to be defined by constant functions.
- Function value must be calculable at elaboration.
 - Usually inputs are constant.
- Greater flexibility for scalable reusable models.
- Multiple limits can be derived from a single constant.

Verilog 2001

```
parameter addw = 5;
parameter datw = 8;

reg [addw-1:0] address;
reg [datw-1:0] mem [1:memsize(addw)];

function integer memsize;
  input [15:0] width;
  begin
    case (width)
      ((width%2)==0): memsize = 1024;
      default:      memsize = 512;
    endcase
  end
endfunction
```

Verilog 1995 module parameter values must be constant expressions, which are limited to operations on literals and previously declared module parameters.

Verilog 2001 permits you to use a constant function call anywhere you are required to use a constant expression.

The standard lists restrictions upon constant function calls:

- Cannot be placed within any generate scope.
- Cannot contain hierarchical references.
- Cannot contain system function calls.
- Ignores system task calls (except will execute \$display in simulator but not elaborator).
- Cannot themselves make constant function calls in any context requiring constant expressions.
 - Can otherwise make constant function calls to functions local to containing module.
- Can access only functions and module parameters and nothing else declared outside the function definition.
- Module parameters must be previously assigned use of defparam can produce undefined results.

Declaring Tasks

- **Task** is declared only within a module.
- Declaration starts with **task** keyword.
- Logic description of what the task is supposed to do follows.
- Ends with **endtask** keyword.
- Alternate syntax with the port list.

```
task zcount;
  input [7:0] in_bus;
  output integer count;
  integer i;
  begin
    count = 0;
    for (i=0; i<=7; i=i+1)
      if (!in_bus[i])
        count = count + 1;
    #5; // mimic latency
  end
endtask
```

Diagram labels for the first code block:

- Name: `zcount`
- Input: `input [7:0] in_bus;`
- Output: `output integer count;`
- Block Item: `integer i;`
- Statement: `begin` block
- Returns value through output: `endtask`

```
task zcount (
  input [7:0] in_bus,
  output [3:0] count );
integer i;
begin
  count = 0;
  for (i=0; i<=7; i=i+1)
    if (!in_bus[i])
      count = count + 1;
  #5; // mimic latency
end
endtask
```

Note ANSI-style port list

7

© Cadence Design Systems, Inc. All rights reserved.



You declare a task only within a module. The declaration starts with the task keyword.

Tasks are by default static. That means that exactly one copy of the task variables exists. All calls to the task utilize that one set of task variables, thus calls to static tasks are generally not re-entrant. External code can access static task variables using hierarchical references. Verilog 2001 added the automatic declaration. Automatic tasks create a new temporary set of their variables for each call. Automatic tasks are re-entrant. External code cannot access automatic task variables.

A task can have any number of input, output or inout ports. You can declare the ports using either syntax. Either syntax accepts port attributes. The port type defaults to a single bit. You can alternatively specify integer, real, realtime, or time, or a vector range. A vector is by default not signed. You can declare the vector signed. The signed declaration is a Verilog 2001 feature.

You can declare additional block items. You cannot declare module items. For example, you can declare variables but not nets.

A task declaration contains a statement. The statement may be a statement block, for example grouped between the begin and end keywords. Tasks can invoke the scheduler. That means task assignments can be blocking or nonblocking and tasks can delay their completion for any amount of time. You can rewrite as a function any task that does not invoke the scheduler.

Tasks can have side effects, for example, a task can assign to a module variable but not to a module net. For a task to have side effects is a very useful feature and common programming practice. A task declaration ends with the endtask keyword.

Calling (Enabling) Tasks

You call (enable) a task as a statement.

```
hierarchical_task_identifier [(expression {,expression})];
```

Why
automatic?

```
task automatic zcount (  
  input [7:0] in_bus,  
  output integer count );  
integer i;  
begin  
  count = 0;  
  for (i=0; i<=7; i=i+1)  
    if (!in_bus[i])  
      count = count + 1;  
  #5; // mimic latency  
end  
endtask
```

```
module zfunc (  
  input [7:0] a_bus, b_bus,  
  output integer acount,  
  output reg bzero  
);  
  
  // task declaration here  
  
  always @(a_bus)  
    zcount(a_bus, acount);  
  
  always @(b_bus) begin: B  
    integer bcount;  
    zcount(b_bus, bcount);  
    bzero = bcount==0;  
  end  
  
endmodule
```

You call a task as a procedural statement. The simulator assigns the values of the input and inout argument expressions to their ports. The standard refers to these “calls” as enables to remind you that a task can consume simulation time and thus can execute concurrently with other procedures.

When the task completes, the simulator assigns the values of the output and inout ports to their lvalue expression arguments. An lvalue expression may appear on the left side of an assignment – you can assign a value to an lvalue expression. If the task never completes, for example, in a loop that loops forever, it never returns and its calling procedure cannot continue. However, if the task “consumes” time, procedures other than the calling procedure can execute.

In this example, the zcount task returns an integer value representing the number of bits of the in_bus port that are zero. It returns a value between 0 and 8. Both task enables are procedural statements. As both procedures can concurrently have the task enabled, the task variables must be automatic variables so that each procedure gets their own copy.

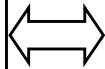
Tasks Without Timing Controls

Tasks can use the scheduler with delays and timing controls, for example.

You can rewrite as a function a task that does not use the scheduler.

Here is such a situation:

```
task zcount (output integer count,  
            input [7:0] in_bus);  
integer i;  
begin  
  count = 0;  
  for (i = 0; i <= 7; i = i + 1)  
    if (!in_bus[i])  
      count = count + 1;  
  end  
endtask
```



```
function integer zcount  
(input [7:0] in_bus);  
integer i;  
begin  
  zcount = 0;  
  for (i = 0; i <= 7; i = i + 1)  
    if (!in_bus[i])  
      zcount = zcount + 1;  
  end  
endfunction
```



Tasks can invoke the scheduler. That means task assignments can be blocking or nonblocking and tasks can delay their completion for any amount of time. A task that does not invoke the scheduler operates very much like a function. You can rewrite as a function any task that does not invoke the scheduler.

Disabling Tasks

You can disable a task (force it to exit):

- In this example, detection of an interrupt disables any currently running *cpu_driver* task.
- Future statements can again enable the *cpu_driver* task.



You can also disable named blocks.

```
module busif_tb;
...
always #50 clk = ~clk;
initial
begin: STIMULUS
    repeat(5) @(negedge clk);
    wait(!interrupt);
    cpu_driver(8'h00);
    wait(!interrupt);
    cpu_driver(8'haa);
    ...
end
...
always @(posedge interrupt)
begin
    disable cpu_driver;
    service_interrupt;
end
endmodule
```



The disable statement terminates activity associated with the task or named block. You can use it to work around the lack of C-like break and continue statements. You can use it to behaviorally model asynchronous activity such as interrupts.

The simulator discontinues any activity associated with the task or named block. If executing the task or named block, it resumes execution with the statement after the task call or after the named block. Implementations can choose whether to remove scheduled nonblocking assignments and whether to discontinue procedural continuous assignments, so this can be a source of differences between simulators.

Disabling a task or named block has no persistence. Subsequent statement execution can immediately again call the task and execution can immediately re-enter the named block, for example, if it is located in an always construct or loop construct.

This example repeatedly calls the *cpu_driver* task to apply stimulus. Upon arrival of an interrupt, this example terminates the current execution of the *cpu_driver* to service the interrupt.

Issues with Functions and Tasks

Following pages address issues presented by functions and tasks:

- Function and task definitions and declarations are by default static.
 - Multiple execution threads within a function or task by default all use the same set of ports and variables.
 - As tasks can consume time, you can easily have multiple task calls simultaneously executing.
 - Functions and tasks can recursively call themselves.
- Function and task arguments are passed by value.
 - Input argument transitions are not visible to the task.
- A function or task can have side effects.
 - Can directly access variables in the scope of the module that defines it.
 - Can access (using out-of-module references) any static variable.



The following pages explore three characteristics of functions and tasks.

- The first issue is that by default there is only one copy of a subroutine's variables, so that multiple outstanding calls to the same subroutine clobber each other.
- The second issue is that arguments are passed and returned by value. The subroutines maintain local variables that get the input values upon invocation. A task cannot observe future transitions on the net or variable arguments from which the passed value was derived.
- The third issue is that subroutines can have side-effects, they can directly access the defining module's nets and variables, and can through out-of-module references access any nets and variables in the simulation. This is not an issue as much as it is a benefit, as it can greatly ease your testbench development effort.

Automatic Tasks: Re-Entering Subroutines

Verilog 1995

- All tasks and functions are static.
 - Concurrent task call and recursive function calls must be avoided.
 - Can cause conflict with internal variables and arguments.

Verilog 2001

- Can declare tasks and functions to be dynamic using keyword `automatic`.
- Allows concurrent task calls and recursive function calls.
 - These declarations cannot be accessed by hierarchical references.
 - These arguments cannot be updated with nonblocking assignment.

Verilog 2001

```
task automatic neg_clocks
    (input [31:0] number_of_edges);
begin
    repeat(number_of_edges)
        @(negedge clk);
end
endtask

initial begin
    neg_clocks(6);
    ...
end

always @(posedge trigger)
begin
    neg_clocks(10);
    ...
end
```

For Verilog 1995, all subroutines are static. Only one copy of a subroutine arguments and variables exists. Multiple concurrent task calls and recursive function calls all use the same copy of inputs, local variables, and outputs.

For Verilog 2001, you can declare an automatic subroutine. A separate copy of the subroutine arguments and variables exists for each invocation. Multiple concurrent task calls and recursive function calls all use their own copy of inputs, local variables, and outputs. As the automatic arguments and variables exist only for the duration of the subroutine execution, you cannot reference them hierarchically from outside the subroutine, cannot assign to them with nonblocking assignments, and cannot use `$monitor` or `$strobe` with them.

Arguments Are Passed by Value

Arguments are passed by value.

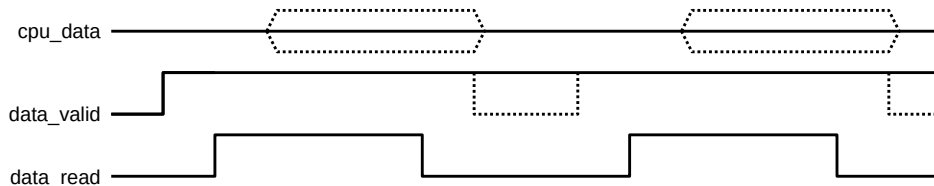
- The simulator assigns expression values to input arguments upon invocation.
- The simulator assigns output values to output argument variables upon return.

Task never completes, -data_valid is never updated, and changes in data_read not detected.



```
task cpu_driver_bad;
(input data_read,
input [7:0] write_data,
output data_valid,
output [7:0] cpu_data);

begin
    #40 data_valid = 1'b1;
    wait(data_read == 1'b1);
    #20 cpu_data = write_data;
    wait(data_read == 1'b0);
    #20 cpu_data = 8'hzz;
    data_valid = 1'b0;
end
endtask
...
cpu_driver_bad(8'hff,data_read,data_valid,cpu_data );
```



13 © Cadence Design Systems, Inc. All rights reserved.

cadence

Subroutine arguments are passed by value.

- The simulator assigns actual argument values to formal input arguments upon invocation.
- The simulator assigns output values to output argument variables upon return.

The problem with the `cpu_driver_bad` task is that the input argument `data_read` is sampled when the task is called and used throughout the task execution. External changes in `data_read` are not seen within the task during the lifetime of the task execution. Therefore the task hangs at the first wait statement.

Also the assignment to the output argument `data_valid` is not made – this will only be updated at the end of the task execution, but because the task never completes, `data_read` is never updated.

Example Task with Side-Effects

Subroutines can have “side effects,” can directly access items of their declaring module, and can access other design items **hierarchically**.

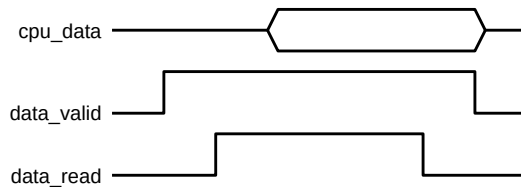
- Advantage – Allows encapsulation of a frequently used algorithm or testbench-DUT transaction.
- Disadvantage – More difficult to re-use the subroutine for other purposes.

```
task cpu_driver_good;
  input [7:0] write_data;
  begin
    #40 data_valid = 1'b1;
    wait (data_read == 1'b1);
    #20 cpu_data = write_data;
    wait (data_read == 1'b0);
    #20 cpu_data = 8'hzz;
    data_valid = 1'b0;
  end
endtask
...
cpu_driver_good(8'hff);
```

data_read, cpu_read, write_data are the variables of declaring module.



Direct references to module variables from within a function or task are resolved to variables within the defining module's scope and not the calling module's scope.



Subroutines can bypass their ports to read and write directly the variables of the defining module, and other design items by hierarchical reference.

This feature greatly simplifies your testbench.

This modified task directly references the `data_valid` and `data_read` module variables instead of having their values passed as argument. The wait statements inside the task can now detect transitions of the `data_read` variable and the task will operate correctly.

Keep in mind that direct subroutine references to module variables are to those within the defining module's scope and not the calling module's scope.

Subroutines Can Access Module Variables

```
module mytasks();

task clockit (input integer n);
  repeat(n) @(negedge busif_tb.clk);
endtask

task cpu_driver;
  input [7:0] write_data;
  begin
    #40 busif_tb.data_valid = 1'b1;
    wait(busif_tb.data_read == 1'b1);
    #20 busif_tb.cpu_data = write_data;
    wait(busif_tb.data_read == 1'b0);
    #20 busif_tb.cpu_data = 8'hzz;
    busif_tb.data_valid = 1'b0;
  end
endtask

...

endmodule
```

Upward
Reference

```
module busif_tb;
  ...
  always #50 clk = ~clk;

  // instantiate tasks module
  mytasks m1 ();

  initial
    begin: STIMULUS
      m1.clockit(5);
      wait(!interrupt);
      m1.cpu_driver(8'h00);
      wait(!interrupt);
      m1.cpu_driver(8'haa);
      ...
    end
  ...
endmodule
```

Downward
Reference



At some point, you can find it convenient to declare a subroutine in one module and call it from another module.

Every Verilog object has a unique path name that starts at a top-level instance, and using the dot (.) character hierarchy separator, traverses down through instance names to the name of the object. You can use these hierarchical names for any Verilog 1995 object. You are not permitted to use the hierarchical name of a Verilog 2001 automatic subroutine variable.

- Hierarchical references can be absolute, starting at a top-level instance and traversing a downward path.
- Hierarchical references can be relative, starting at the scope of the reference and traversing a downward path.
- Hierarchical references can also be upward, but such references can have only a single instance or module name, which resolves to the nearest upward instance or module with that name.

Module Summary

You can now make your code more readable and reusable by encapsulating functionality in Verilog subroutines.

This module describes:

- Subroutine concepts:
 - Encapsulate code that might otherwise be duplicated.
- Functions:
 - Have one or more inputs and return a single value through their name.
 - Are invoked as an expression term.
- Tasks:
 - Have zero or more input and zero or more outputs.
 - Are invoked as a procedural statement.
- Issues:
 - Subroutine variables are by default static.
 - Subroutine arguments are passed by value.
 - Subroutines can directly access module nets and variables.



After completing this module, you can effectively use subroutines to encapsulate functionality and make your code more readable and reusable. This module described Verilog subroutines known as functions and tasks. It explained where and how to define them, how to invoke them, and some issues involved with using them.

Module Review

1. Which subroutine(s) can contain timing controls?
2. A call to which subroutine(s) can appear outside procedures?
3. What is the default type of a subroutine port?
4. By which method are subroutine arguments passed (value, pointer, reference)?



This page does not contain notes.

Module Review Solutions

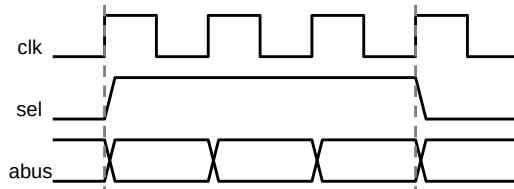
1. Which subroutine(s) can contain timing controls?
 - Only a task can contain timing controls. A function cannot invoke the scheduler in any way.
2. A call to which subroutine(s) can appear outside procedures?
 - You use a function call as a term of an expression, so a function call can appear on the right side of a continuous assignment.
3. What is the default type of a subroutine port?
 - A subroutine port is by default a single-bit reg. You can declare it as any variable type. Subroutines cannot declare nets.
4. By which method are subroutine arguments passed (value, pointer, reference)?
 - Subroutine arguments are passed by value.



This page does not contain notes.

Module Exercise

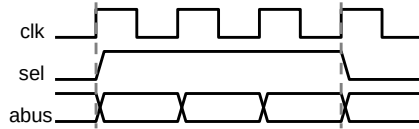
Write a task to accept a 32-bit input argument address and place that address on the “abus” and increment the address through a total of four cycles. The task also controls the “sel” signal. Refer to the diagram.



This page does not contain notes.

Module Exercise Solution

Write a task to accept a 32-bit input argument address and place that address on the “abus” and increment the address through a total of four cycles. The task also controls the “sel” signal. Refer to the diagram.



Solution:

```
task task4;
  input [31:0] addr;
  begin
    @(posedge clk)
      sel <= 1;
      abus <= addr;
    repeat(3)
      @(posedge clk)
        abus <= abus + 1;
      sel <= 0;
    end
  endtask
```



This page does not contain notes.



Module 13-2

Directing the Compiler

cadence®

This module describes the Verilog compiler directives.

A compiler directive is a directive to the compiler. It is not syntactically a Verilog declaration or a Verilog statement. The scope of a compiler directive is from the point in the source stream input to the compiler to the point at which the directive is overridden or disabled or the end of the compilation session, probably across multiple files and multiple module descriptions, and sometimes affecting only parts of files or parts of module descriptions.

Verilog compiler directives are prefixed with the “back-tick” (‘) character more formally called the accent grave. Wherever you see that character in Verilog code it is always associated with a compiler directive.

This module introduces compiler directives here. Although they are more closely related to lexical structure than to declarations and statements, you will understand them better if you have an understanding of Verilog declarations and statements. The course will later revisit many of these directives in an appropriate context that uses them.

Module Objective

In this module, you:

- Direct the compiler to interpret subsequent source code

Topics

- Substituting text
- Conditionally compiling code
- Including files
- Setting the timescale
- Reserving keywords
- Using pragmas



Your objective is to appropriately and effectively utilize compiler directives. To do that, you need to know what directives are available and what they do and how to use them.

Substituting Text: The `define Directive

Define a text substitution macro:

- ``define name[(arguments)] text`
- Text can use other text macros.

Use a text substitution macro:

- ``name[(arguments)]`
- Preprocessor substitutes text literally (WYSIWYG).

Undefine a text macro:

- ``undef name`

```
`define width 7
wire a, b, c;
reg [2:0] f;
reg ['width:0] mem [1:1024];

`define e {b, c, a};

always @ (posedge clk)
    f<= `e ; //compiler replaces macro
           // with its definition
```



You can use the text substitution facility anywhere that you want to write generic code that you can then easily replace during subsequent compilation sessions without the need to edit the target source file. You can define the text substitution macros in a separate file that you compile first during the compilation session, and many tools also provide a vendor-dependent command-line way to define text substitution macros.

As the scope of a compiler directive depends upon compilation order and thus can change between compilation sessions, you may want to instead use module parameters wherever possible for design constants such as delays and vector widths.

You can place a `define directive anywhere in your source code, but most people by convention place them all in a separate file, or at the top of the file if they are used in only one file.

- Text macros have their own name space. You can define a macro with the same name as a module without causing confusion. You cannot name the text macro with the same name as an existing compiler directive, as that would definitely cause confusion.
- You can include a parenthesized list of formal arguments. The opening parenthesis must appear immediately after the macro identifier with no intervening white space.
- A newline character terminates the macro definition. You can escape the newline with a backslash (\) character to continue the definition on a subsequent line.
- Some constructs you cannot split across text macros. This means that if a macro definition starts the construct then it must complete the construct in that same macro definition. Constructs you cannot split across text macros are comments, identifiers, keywords, numbers, operators and strings.

Conditionally Compiling Code: The `ifdef Directive

Conditionally compile code regions:

- Verilog 1995: `ifdef, `else, `endif
- Verilog 2001: `ifndef, `elsif
- You can define the macros without providing a value
- Many tools provide a vendor-dependent command-line way to set text replacement macros

```
localparam integer SIZE = 256;
...
reg [31:0] mem [1:SIZE];
integer i;
initial
  for (i = 1; i <= SIZE; i = i + 1)
    `ifdef init2zero
      mem[i] = 0;
    `else
      mem[i] = i;
    `endif
```



You can use the existence of a text macro to conditionally compile lines of source code.

You use these directives similarly in an algorithmic sense to the if-else statement, but of course keep in mind that these macros must each appear on their own line and it is the preprocessor that parses them during compile time.

This example provides the option to initialize the ROM to either all zeros or to values read from a file.

Including Files: The ``include` Directive

globals.txt

```
// Clock and simulator constants
// could also include tasks etc.
localparam PERIOD = 10;
localparam POSEDGE_FIRST = 1;
localparam START_TIME = 2;
localparam NUMBER_CYCLES = 100;
```

test.v

```
...
// Include global variables
`include "globals.txt"
reg clk;
initial
begin : CLKGEN
  clk = ~POSEDGE_FIRST;
  #START_TIME
  repeat(2*NUMBER_CYCLES)
    #(PERIOD/2) clk = ~clk;
  $display($time, "Test Done");
  $finish;
end
```

Insert the contents of a source file:

```
`include "filename"
```

- Use included files to ensure that team-wide module descriptions use the same declarations:
 - Constants
 - Tasks and functions
- The included file can contain another ``include` directive:
 - The standard guarantees nesting up to 15 deep.

The double-quoted argument is a literal string so cannot be a text macro.



The ``include` directive includes the contents of a file at the point where the directive appears. The effect is exactly as if the file contents were copied into the source at that point to replace the ``include` directive.

People use this directive to include declarations that are used by all team members and so for the sake of consistency should be maintained in one place.

The standard guarantees that you can nest included files at least 15 deep.

The argument to the ``include` directive is a quoted file path name and not a compiler directive or string literal. No standard way exists to generate the file name to be included.

Include File Search Path Command Option: `-incdir`

An include file may be defined in a different directory than where it is included

A ``include` string can be a pathname to a file

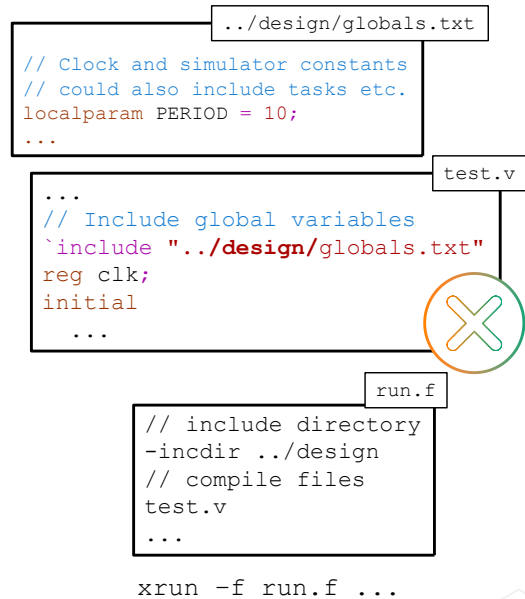
- This breaks reusability by enforcing a specific directory structure!

You could use an environmental variable..

- .. but this still requires the variable to be set

Solution is to use the `incdir` command line option

- `-incdir ../design`
- Defines a search path for resolution of ``include`
- Multiple `incdir` lines are allowed
- Directory path is not a string!
- Place in a run file for flexibility



This page does not contain notes.

Setting the Timescale: The `timescale Directive

Set time unit and time precision:

``timescale unit / precision`

- 1 or 10 or 100 followed by unit.
- *unit* is one of fs, ps, ns, us, ms, s.
- *precision* cannot be larger than unit.
- The simulator rounds time specifications to the precision of the module and scales them to the time unit of the module.
- The overall simulation precision is the smallest of the defined precisions.

Important: ``timescale` must appear outside of the module.

```
`timescale 10ns/1ns
module test;
  localparam real delay = 2.55;
  ...
  #delay; // 26 ns delay
  ...
endmodule
```

```
`timescale 1ns/100ps
module first (...);
  ...
  #10; // 10 ns delay
  ...
endmodule
```

```
`timescale 1ps/100fs
module second (...);
  ...
  #100; // 100 ps delay
  ...
endmodule
```

The ``timescale` directive specifies the time unit and time precision for subsequent module declarations. Any expression used in a context to mean simulation time will assume the time unit of the current module and be rounded to the time precision of the current module. System tasks that display simulation time will scale the display to the time unit of the current module.

As the standard does not prescribe a default time scale, for interoperability either all modules must be subject to a time scale or no module can be subject to a time scale. To ensure that all modules are subject to a time scale, you should get used to specifying a time scale at the top of every file, even if you not use time specifications in the file. To modify a time scale, you need to edit the file and recompile that file and all other files that utilized that directive. No method exists to modify a time scale during simulation.

In this example, the file containing the test module specifies that until further notice all subsequent time specifications are in units of 10ns and the time precision is 1ns. The 2.55 real value when used as a delay then means 25.5ns and is rounded to 26ns. The file containing the “second” module specifies that until further notice all subsequent time specifications are in units of 1ps and the time precision is 100fs. As this is the finest precision that any of these modules uses, if no other module requires a finer precision, this is the precision of the simulation.

Default Command Line Timescale -timescale

``timescale` is a compiler directive

- Remains in effect across multiple files until redefined or undefined
- Causes file order dependencies in compilation

Cadence Xcelium simulator forces a solution to this:

- If there is a timescale on **one** module, there must be a timescale on **every** module
- Modules *without* an explicit timescale generate NOTSET elaboration errors

A command line option `-timescale` defines a default timescale for all modules without an explicit

- `xrun -timescale 1ns/10ps ...`
- Avoids NOTSET errors

```
xrun(64): ...
Elaborating the design hierarchy:
Top level design units:
...
Modules w/wo timescales:
  timescale of worklib.fifo:v = 1ns/100ps
  timescale of worklib.port_fsm:v = NOTSET
  timescale of worklib.hw_top:v = NOTSET
  timescale of worklib.host_ctl:v = NOTSET
...
xmelab: *F,CUMSTS: Timescale directive missing
on one or more modules.
xrun: *E,ELBERR: Error (*E) ...
```

timescale on fifo.v, but not on other modules, causes NOTSET errors on modules without explicit timescale

As ``timescale` is a compiler directive, it applies from the point when it is executed until it is redefined, undefined or the last file has been compiled. This makes the timescale file order dependent, i.e. if you compile files in a different order, modules can inherit a different timescale and so behave differently in simulation.

To address this problem, most simulators modify the behavior of the timescale directive. For interoperability, either **all** modules must be subject to a time scale or **no** module can be subject to a time scale.

If only **some** of the modules have an explicit timescale defined, then the Xcelium elaboration step will generate NOTSET errors on the modules that do not have explicit timescales.

To ensure that all modules are subject to a time scale, you could specify a time scale at the top of **every** file, even if you do not use time specifications in the file. To modify a time scale, you need to edit the file and recompile that file and all other files that utilized that directive. No method exists to modify a time scale during simulation.

This is not ideal, so as a solution, most simulators allow you to use a command line option to define a default timescale that will apply to all modules which do not have an explicit timescale. In fact, it is common practice to solely use the command line timescale option, as this ensures a consistent timescale across all modules and can be changed without modifying design files.

Reserving Keywords: `begin\_keywords, `end_keywords

Reserve keywords for identifiers:

```
`begin_keywords "version_spec"  
`end_keywords
```

- Verilog-2005 directive for reserving keywords for user identifiers
- Place outside design element
 - **config, module, primitive**
- Version specifier:
 - 1364-1995
 - 1364-2001-noconfig
 - 1364-2001
 - 1364-2005

```
`begin_keywords "1364-1995"  
module m1995 (...);  
    task generate;  
        ...  
    endtask  
    ...  
endmodule  
`end_keywords
```

generate is a keyword
new to Verilog-2001

```
`begin_keywords "1364-2001-noconfig"  
module design (...);  
    ...  
endmodule  
`end_keywords
```

design is a configuration
keyword new to Verilog-2001

```
`begin_keywords "1364-2001"  
module m2001 (...);  
    wire unsigned uwire;  
    ...  
endmodule  
`end_keywords
```

uwire is a keyword
new to Verilog-2005

The ``begin_keywords` directive tells a parser to reserve for user identifiers all new keywords defined by a later standard. These keywords can appear only outside design elements, such as outside configuration, module, or primitive declarations.

If you have a legacy module that uses the `generate` word as a user identifier, then you can direct a Verilog 2005 compliant compiler to not use any keywords more recent than the 1995 standard for that module.

If you have a legacy module that uses the `design` word as a user identifier, you can direct a Verilog 2005 compliant compiler to not use any Verilog 2001 configuration keywords or any other keywords more recent than the 2001 standard for that module.

If you have a legacy module that uses the `uwire` word as a user identifier, you can direct a Verilog 2005 compliant compiler to not use any keywords more recent than the 2001 standard for that module.

Keywords new to IEEE Std 1364-2001

```
automatic cell config design endconfig endgenerate generate genvar incdir  
include instance liblist library localparam noshowcancelled  
pulsestyle_oneevent pulsestyle_ondetect showcancelled signed unsigned use
```

Keywords new to IEEE Std 1364-2005

```
uwire
```

Using Pragmas: The `pragma Directive

Utilize a pragma:

```
`pragma pragma_name  
[ pragma_expression  
{, pragma_expression } ]
```

- Verilog-2005 structure for implementation-specific directives
- Verilog defines just a few:
 - `pragma reset
 - `pragma resetall
 - `pragma protect

```
module smux  
#(parameter integer W=1)  
(output [W-1:0] y, input s,  
input [W-1:0] a, b);  
`pragma protect begin  
assign y = s ? b : a ;  
`pragma protect end  
endmodule
```

Area to protect

The `pragma directive provides a means for implementations to extend the set of compiler directives. A pragma name follows the directive and is followed by an optional list of pragma expressions. A pragma expression is a pragma keyword or a pragma value or an assignment of a pragma value to a pragma keyword.

The Verilog 2005 update defines only a few pragmas:

- The reset pragma resets the pragmas provided as pragma expressions;
- The resetall pragma resets all pragmas; and
- The protect pragma encrypts subsequent source code.

An implementation ignores pragmas it does not recognize.

```
pragma ::=  
  `pragma pragma_name [ pragma_expression { , pragma_expression } ]  
pragma_name ::= simple_identifier  
pragma_expression ::=  
  pragma_keyword  
| pragma_keyword = pragma_value  
| pragma_value  
pragma_keyword ::= simple_identifier  
pragma_value ::=  
  ( pragma_expression { , pragma_expression } )  
| number  
| string  
| identifier
```

Disabling Implicit Net Declarations

``default_nettype`

New default net type: `none`

- Verilog-1995 permits implicit net declarations.
 - Use a previously undeclared identifier in a port expression or terminal list.
 - Becomes the default net type (initially a wire).
 - You change the default net type with ``default_nettype`.
 - `tri tri0 tril triand trior trireg wand wire wor`
- Verilog-2001 adds the default net type `none`.
 - Set this as the default net type with ``default_nettype none`.
 - Undeclared signals become a syntax error.
 - Reduces potential for typographical error.



You implicitly declare a net when you use a previously undeclared identifier in a port expression or terminal list or as the lvalue of a continuous assignment. That net by default becomes a wire. You set the value of the ``default_nettype` directive to change that default net type. The Verilog 2001 standard adds the `none` type. When you set the default net type to `none`, you require explicit declaration of all nets, thus reducing the potential for typographical error.

Note: All Verilog compiler directives start with the grave accent (sometimes called a “back-tick”). You should be aware that some fonts cannot render this character properly and display it as an acute accent (a “forward tick”).

Reference: Compiler Directives

<code>`celldefine</code> <code>`endcelldefine</code>	Tags a library cell
<code>`default_nettype</code>	Sets the net type for implicit net declarations
<code>`define</code> <code>`undef</code>	Defines and undefines a text macro
<code>`ifdef</code> <code>`else</code> <code>`endif</code>	Conditionally compiles code depending upon text macro existence
<code>`include</code>	Includes a source file
<code>`resetall</code>	Resets directives to their initial state
<code>`timescale</code>	Sets the time units and time precision
<code>`unconnected_drive</code> <code>`nounconnected_drive</code>	Pulls up/pulls down unconnected module inputs
<code>`ifndef</code> <code>`elsif</code>	Verilog-2001: More conditional compilation
<code>`line</code>	Verilog-2001: Overrides reported source file and line upon error
<code>`begin_keywords</code> <code>`end_keywords</code>	Verilog-2005: Reserves keywords for use as identifiers
<code>`pragma</code>	Verilog-2005: Changes how compiler interprets subsequent source

32 © Cadence Design Systems, Inc. All rights reserved.



Use the ``celldefine` and ``endcelldefine` directives to tag modules as technology cells. PLI applications such as delay calculators and power calculators query this tag to determine the design hierarchy leaf cells that have a physical implementation.

Use the ``default_nettype` directive to set the default net type for implicitly declared nets. This is by default the wire type but you can set it to any net type. The Verilog 2001 standard adds the argument `none` which prevents implicitly declared nets.

Use the ``define` directive to define a text replacement macro and the ``undef` directive to undefine it.

Use the ``ifdef`, ``else` and ``endif` directives to conditionally compile source text depending upon the existence of a text macro. The Verilog 2001 standard adds the ``ifndef` and ``elsif` directives.

Use the ``include` directive to insert the contents of a source file at the point of the directive. The argument to the directive is a quoted file name. No standard way exists to generate the file name.

Use the ``resetall` directive to reset all compiler directives back to their initial state. This is very useful and recommended for placing at the beginning of every source file to ensure that only the directives set by that file are in effect during parsing of the file. This directive affects the ``celldefine`, ``default_nettype`, ``timescale` and ``unconnected_drive` directives. It does not undefine text macros, so your file can still be dependent upon text macros defined in some other file.

Use the ``timescale` directive to specify the time unit and time precision for subsequent module declarations. Any expression used in a context to mean simulation time will assume the time unit of the current module and be rounded to the time precision of the current module. System tasks that display simulation time will scale the display to the time unit of the current module.

Use the ``pragma` directive to change how the compiler interprets subsequent source. A pragma consists of a name and an optional keyword and an optional value. The standard defines only the `reset` and `resetall` keywords. Tool vendors define additional pragmas.

Module Summary

You should now be able to direct the compiler how to interpret subsequent source code.

This module described:

- The **`define** and **`undef** directives for substituting text.
- The **`ifdef**, **`else**, **`endif**, **`ifndef**, and **`elsif** directives for conditionally compiling code.
- The **`include** directive for including files.
- The **`timescale** directive for setting the timescale.
- The **`begin\_keywords** and **`end_keywords** directives for reserving keywords.
- The **`pragma** directive for using pragmas.
- The **default_nettype** directive from Verilog-2001.



Your objective is to appropriately and effectively utilize compiler directives. To do that, you need to know what directives are available and what they do and how to use them.